

OkGo Android网络请求库使用指南

OkGo 是基于 OkHttp 封装的 Android 网络请求库，简化了 OkHttp 的使用流程，支持 GET/POST、文件上传/下载、断点续传、缓存、请求取消等常用功能，上手简单且扩展性强。下面从**核心使用步骤**、**常用请求示例**、**关键特性**三部分讲解：

一、核心使用步骤

1. 集成依赖

在项目 `build.gradle`（Module 级别）中添加依赖（需确保 OkHttp 版本兼容）：

Code block

```
1  // 核心库
2  implementation 'com.lzy.net:okgo:3.0.4'
3  // 可选：如果需要解析 JSON（搭配 Gson）
4  implementation 'com.lzy.net:okrx2:3.0.4'
5  implementation 'com.google.code.gson:gson:2.10.1'
```

注意：最新版本可参考 [OkGo 官方仓库](#)。

2. 初始化（全局仅需一次）

在 `Application` 的 `onCreate()` 中初始化：

Code block

```
1  public class MyApp extends Application {
2      @Override
3      public void onCreate() {
4          super.onCreate();
5          // 基本初始化
6          OkGo.getInstance().init(this)
7              // 可选配置：超时时间、拦截器、缓存等
8              .setConnectTimeout(OkGo.DEFAULT_MILLISECONDS) // 连接超时（默认10
秒)
9              .setReadTimeOut(OkGo.DEFAULT_MILLISECONDS)    // 读取超时
10             .setWriteTimeOut(OkGo.DEFAULT_MILLISECONDS);   // 写入超时
11     }
12 }
```

记得在 `AndroidManifest.xml` 中注册 `Application`，并添加网络权限：

Code block

```
1 <uses-permission android:name="android.permission.INTERNET" />
2 <application
3     android:name=".MyApp"
4     ...>
5 </application>
```

二、常用请求示例

OkGo 采用链式调用，核心入口是 `OkGo.<T>get/post()`，泛型 `T` 为返回数据类型（如 `String`、自定义 `Bean`）。

1. GET 请求（最基础）

Code block

```
1 // 示例：获取字符串数据
2 OkGo.<String>get("https://www.example.com/api/data") // 请求地址
3     .params("page", 1) // 添加参数（键值对）
4     .params("size", 10)
5     .execute(new StringCallback() { // 回调（StringCallback 对应返回String）
6         @Override
7         public void onSuccess(Response<String> response) {
8             // 请求成功：response.body() 为返回的字符串
9             String data = response.body();
10            Log.d("OkGo", "GET成功: " + data);
11        }
12
13        @Override
14        public void onError(Response<String> response) {
15            // 请求失败：可获取错误信息
16            Log.e("OkGo", "GET失败: " + response.getException().getMessage());
17        }
18    });
```

2. POST 请求（表单提交）

Code block

```
1 // 示例：POST提交表单，返回自定义Bean（需Gson解析）
2 OkGo.<UserBean>post("https://www.example.com/api/login")
3     .params("username", "test") // 表单参数
4     .params("password", "123456")
```

```

5      .execute(new JsonCallback<UserBean>() { // JsonCallback 自动解析为Bean
6          @Override
7          public void onSuccess(Response<UserBean> response) {
8              UserBean user = response.body();
9              Log.d("OkGo", "登录成功: " + user.getNickname());
10         }
11
12         @Override
13         public void onError(Response<UserBean> response) {
14             Log.e("OkGo", "登录失败: " + response.getException().getMessage());
15         }
16     });

```

自定义 Bean 示例:

Code block

```

1  public class UserBean {
2      private String nickname;
3      private String token;
4      // 生成get/set方法
5  }

```

3. 文件下载（支持断点续传）

Code block

```

1  String downloadUrl = "https://www.example.com/file/apk/test.apk";
2  String savePath = Environment.getExternalStorageDirectory() + "/OkGoDownload/";
3  String saveFileName = "test.apk";
4
5  OkGo.<File>get(downloadUrl)
6      .execute(new FileCallback(savePath, saveFileName) { // 文件保存路径+文件名
7          @Override
8          public void onSuccess(Response<File> response) {
9              // 下载成功: response.body() 为文件对象
10             Log.d("OkGo", "下载完成: " + response.body().getAbsolutePath());
11         }
12
13         @Override
14         public void onError(Response<File> response) {
15             Log.e("OkGo", "下载失败: " + response.getException().getMessage());
16         }
17
18         @Override
19         public void downloadProgress(Progress progress) {

```

```

20         // 下载进度: progress.fraction 为0-1的进度值
21         float progressValue = progress.fraction;
22         Log.d("OkGo", "下载进度: " + (progressValue * 100) + "%");
23     }
24 }

```

注意：需添加文件读写权限（Android 6.0+ 需动态申请）：

Code block

```

1 <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
2 <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />

```

4. 文件上传（单文件/多文件）

Code block

```

1 // 示例: 上传图片+表单参数
2 String uploadUrl = "https://www.example.com/api/upload";
3 File imageFile = new File(Environment.getExternalStorageDirectory() +
4     "/test.jpg");
5 OkGo.<String>post(uploadUrl)
6     .params("userId", "123") // 普通表单参数
7     .params("file", imageFile) // 上传文件 (key为接口要求的字段名)
8     .execute(new StringCallback() {
9         @Override
10         public void onSuccess(Response<String> response) {
11             Log.d("OkGo", "上传成功: " + response.body());
12         }
13
14         @Override
15         public void uploadProgress(Progress progress) {
16             // 上传进度
17             float progressValue = progress.fraction;
18             Log.d("OkGo", "上传进度: " + (progressValue * 100) + "%");
19         }
20     });

```

三、关键特性与注意事项

1. 请求取消：可通过 `tag` 标记请求，批量/单个取消：

Code block

```
1 // 请求时设置tag
2 OkGo.get(url).tag("tag1").execute(...);
3 // 取消单个请求
4 OkGo.getInstance().cancelTag("tag1");
5 // 取消Activity/Fragment所有请求（绑定生命周期）
6 OkGo.getInstance().cancelTag(this);
```

2. 缓存机制：支持自定义缓存策略（如仅缓存、优先缓存等）：

Code block

```
1 OkGo.<String>get(url)
2     .cacheMode(CacheMode.FIRST_CACHE_THEN_REQUEST) // 先读缓存，再请求刷新
3     .cacheKey("cache_key_1") // 缓存key（唯一标识）
4     .execute(...);
```

3. 生命周期绑定：避免内存泄漏，可绑定 Activity/Fragment 生命周期：

Code block

```
1 OkGo.get(url)
2     .tag(this) // 绑定当前页面
3     .retryCount(3) // 失败重试次数
4     .execute(...);
```

4. 线程处理：回调默认在主线程（可直接更新UI），无需手动切换线程。

四、总结

OkGo 的核心优势是**封装度高、API 简洁**，基于 OkHttp 保留了高性能，同时解决了 OkHttp 原生使用繁琐的问题。日常开发中，GET/POST 满足大部分接口需求，文件上传/下载场景也能快速实现，适合快速开发。

注意：如果项目追求更轻量、更灵活的封装，也可直接使用 OkHttp + Retrofit；OkGo 适合需要快速落地、减少重复封装的场景。

（注：文档部分内容可能由 AI 生成）